

```
In [2]: import pandas as pd

df = pd.read_csv("dataset.csv")

print("Rows:", df.shape[0])
print("Columns:", df.shape[1])

df.head()
```

Rows: 1048
Columns: 33

Out[2]:

	projectName	state	city	county	region	status	sponsors	operators
0	Meta Montgomery Data Center	Alabama	Montgomery	Montgomery County	Central Alabama	under- construction	Meta Platforms, Inc.	Meta Platforms, Inc.
1	Meta Huntsville Data Center	Alabama	Toney (Huntsville area)	Madison County	Northern Alabama	operational	Meta Platforms, Inc.	Meta Platforms, Inc.
2	Google Bridgeport Data Center	Alabama	Bridgeport	Jackson County	Northern Alabama	expansion	Google LLC	Google LLC
3	Project Marvel - Bessemer Hyperscale Data Center	Alabama	Bessemer	Jefferson County	Central Alabama	planned	QTS Data Centers; Logistic Land Investment LLC...	NaN
4	DC BLOX Montgomery Data Center	Alabama	Montgomery	Montgomery County	Central Alabama	under- construction	DC BLOX	DC BLOX

5 rows × 33 columns

```
In [3]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1048 entries, 0 to 1047
Data columns (total 33 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   projectName                          1048 non-null   object
1   state                                1048 non-null   object
2   city                                 1048 non-null   object
3   county                              1048 non-null   object
4   region                              588 non-null    object
5   status                              1048 non-null   object
6   sponsors                             1039 non-null   object
7   operators                           1008 non-null   object
8   tenants                             227 non-null    object
9   totalSquareFeet                     758 non-null    float64
10  powerCapacityMW                     861 non-null    float64
11  itLoadMW                             183 non-null    float64
12  investmentUSD                        707 non-null    float64
13  sizeNotes                           530 non-null    object
14  announcedDate                       1002 non-null   object
15  constructionStartDate               370 non-null    object
16  expectedCompletionDate              668 non-null    object
17  actualCompletionDate                219 non-null    object
18  numberOfPhases                     607 non-null    float64
19  purpose                             1017 non-null   object
20  renewableEnergy                    344 non-null    object
21  pueTarget                           47 non-null     float64
22  waterCooling                       461 non-null    object
23  certifications                      183 non-null    object
24  sustainabilityNotes                602 non-null    object
25  economicImpact_jobs                269 non-null    float64
26  economicImpact_taxRevenue          118 non-null    float64
27  economicImpact_notes               625 non-null    object
28  incentives_type                     363 non-null    object
29  incentives_value                    102 non-null    float64
30  challenges                          630 non-null    object
31  notes                              1048 non-null   object
32  lastUpdated                        1048 non-null   object
dtypes: float64(9), object(24)
memory usage: 270.3+ KB
```

```
In [4]: # =====
# BLOCK 1 – UNDERSTAND THE DATA
# =====

# 1. Dataset dimensions
print("DATASET DIMENSIONS")
print("-----")
print(f"Projects: {df.shape[0]}")
print(f"Variables: {df.shape[1]}")

# 2. Column names
print("\nVARIABLES")
print("-----")
for i, col in enumerate(df.columns, start=1):
    print(f"{i}. {col}")

# 3. Data types
print("\nDATA TYPES")
print("-----")
print(df.dtypes)
```

```
# 4. Missingness
missing = pd.DataFrame({
    "missing_count": df.isna().sum(),
    "missing_percent": (df.isna().mean() * 100).round(1)
})

missing = missing.sort_values(
    "missing_percent",
    ascending=False
)

print("\nMISSING DATA")
print("-----")
display(missing)

# 5. Critical variables for our research question
critical = [
    "projectName",
    "status",
    "powerCapacityMW",
    "itLoadMW",
    "investmentUSD",
    "numberOfPhases",
    "purpose",
    "region",
    "announcedDate"
]

print("\nCRITICAL VARIABLES")
print("-----")
display(df[critical].head(20))

# 6. Candidate sample with both investment and power capacity
has_investment = df["investmentUSD"].notna()
has_capacity = df["powerCapacityMW"].notna()
has_both = has_investment & has_capacity

print("\nCANDIDATE SAMPLE")
print("-----")
print(f"Total projects: {len(df)}")
print(f"Projects with investment: {has_investment.sum()}")
print(f"Projects with power capacity: {has_capacity.sum()}")
print(f"Projects with BOTH: {has_both.sum()}")
print(f"Share with both: {has_both.mean() * 100:.1f}%")
```

DATASET DIMENSIONS

Projects: 1048

Variables: 33

VARIABLES

1. projectName
2. state
3. city
4. county
5. region
6. status
7. sponsors
8. operators
9. tenants
10. totalSquareFeet
11. powerCapacityMW
12. itLoadMW
13. investmentUSD
14. sizeNotes
15. announcedDate
16. constructionStartDate
17. expectedCompletionDate
18. actualCompletionDate
19. numberOfPhases
20. purpose
21. renewableEnergy
22. pueTarget
23. waterCooling
24. certifications
25. sustainabilityNotes
26. economicImpact_jobs
27. economicImpact_taxRevenue
28. economicImpact_notes
29. incentives_type
30. incentives_value
31. challenges
32. notes
33. lastUpdated

DATA TYPES

projectName	object
state	object
city	object
county	object
region	object
status	object
sponsors	object
operators	object
tenants	object
totalSquareFeet	float64
powerCapacityMW	float64
itLoadMW	float64
investmentUSD	float64
sizeNotes	object
announcedDate	object
constructionStartDate	object
expectedCompletionDate	object
actualCompletionDate	object
numberOfPhases	float64
purpose	object
renewableEnergy	object

pueTarget	float64
waterCooling	object
certifications	object
sustainabilityNotes	object
economicImpact_jobs	float64
economicImpact_taxRevenue	float64
economicImpact_notes	object
incentives_type	object
incentives_value	float64
challenges	object
notes	object
lastUpdated	object
dtype:	object

MISSING DATA

	missing_count	missing_percent
pueTarget	1001	95.5
incentives_value	946	90.3
economicImpact_taxRevenue	930	88.7
certifications	865	82.5
itLoadMW	865	82.5
actualCompletionDate	829	79.1
tenants	821	78.3
economicImpact_jobs	779	74.3
renewableEnergy	704	67.2
incentives_type	685	65.4
constructionStartDate	678	64.7
waterCooling	587	56.0
sizeNotes	518	49.4
region	460	43.9
sustainabilityNotes	446	42.6
numberOfPhases	441	42.1
economicImpact_notes	423	40.4
challenges	418	39.9
expectedCompletionDate	380	36.3
investmentUSD	341	32.5
totalSquareFeet	290	27.7
powerCapacityMW	187	17.8
announcedDate	46	4.4
operators	40	3.8
purpose	31	3.0
sponsors	9	0.9
notes	0	0.0
projectName	0	0.0
state	0	0.0
status	0	0.0
county	0	0.0
city	0	0.0
lastUpdated	0	0.0

CRITICAL VARIABLES

	projectName	status	powerCapacityMW	itLoadMW	investmentUSD	numberOfPhases
0	Meta Montgomery Data Center	under- construction	195.00	NaN	1.500000e+09	2.0
1	Meta Huntsville Data Center	operational	214.00	NaN	1.500000e+09	2.0
2	Google Bridgeport Data Center	expansion	143.00	NaN	2.000000e+09	0.0
3	Project Marvel - Bessemer Hyperscale Data Center	planned	1200.00	NaN	1.450000e+10	0.0
4	DC BLOX Montgomery Data Center	under- construction	40.00	NaN	3.000000e+07	0.0
5	DC BLOX Huntsville Expansion	expansion	5.00	5.000	3.500000e+07	0.0
6	DC BLOX Birmingham Data Center	operational	5.00	49.000	7.850000e+08	0.0
7	DC BLOX Huntsville (Existing)	operational	13.50	9.000	1.300000e+07	0.0
8	AT&T Hoover Data Center Upgrade	operational	NaN	NaN	4.000000e+07	0.0
9	Southern Company Birmingham Data Center	operational	3.00	NaN	2.000000e+07	0.0
10	University of Alabama High Performance Computi...	under- construction	14.60	14.600	9.600000e+07	0.0
11	Far North Digital Prudhoe Bay Data Center	planned	120.00	NaN	NaN	1.0
12	GCI Anchorage Data Center	operational	NaN	NaN	NaN	0.0
13	Greensparc Cordova Edge Data Center	expansion	0.15	NaN	NaN	0.0
14	Hilcorp Endicott	planned	1.40	1.400	NaN	0.0

	projectName	status	powerCapacityMW	itLoadMW	investmentUSD	numberOfPhases
	Bitcoin Mining Pilot					
15	Microsoft West US 3 Azure Region	expansion	NaN	48.327	1.500000e+09	0.0
16	Meta Mesa Data Center	operational	450.00	NaN	1.000000e+09	2.0
17	Google Mesa Data Center (Redhawk)	expansion	360.00	NaN	6.000000e+08	2.0
18	Vantage Data Centers Phoenix Campus	under-construction	160.00	176.000	1.500000e+09	0.0
19	Vermaland La Osa Data Center Park	planned	1000.00	NaN	3.300000e+10	0.0

CANDIDATE SAMPLE

Total projects: 1048
Projects with investment: 707
Projects with power capacity: 861

```
In [6]: # =====  
# BLOCK 2 – ELIGIBILITY GATE 1: NUMERICAL VALIDITY  
# =====  
  
cost_data = df[  
    (df["investmentUSD"].notna())  
    & (df["investmentUSD"] > 0)  
    & (df["powerCapacityMW"].notna())  
    & (df["powerCapacityMW"] > 0)  
].copy()  
  
# Candidate target  
cost_data["investmentPerMW"] = (  
    cost_data["investmentUSD"]  
    / cost_data["powerCapacityMW"]  
)  
  
print("ELIGIBILITY GATE 1")  
print("-----")  
print(f"Raw projects: {len(df):,}")  
print(f"Eligible projects: {len(cost_data):,}")  
print(f"Excluded projects: {len(df) - len(cost_data):,}")  
print(f"Coverage: {len(cost_data) / len(df) * 100:.1f}%")  
  
print("\nCAPITAL INTENSITY")  
display(cost_data["investmentPerMW"].describe())
```

ELIGIBILITY GATE 1

Raw projects: 1,048

Eligible projects: 616

Excluded projects: 432

Coverage: 58.8%

CAPITAL INTENSITY

count 6.160000e+02

mean 1.048611e+07

std 1.866486e+07

min 3.200000e+04

25% 4.492424e+06

50% 7.000000e+06

75% 1.078297e+07

max 3.344482e+08

Name: investmentPerMW, dtype: float64

```

In [7]: # ATLAS RCF v0.1 – Model Specification

## Target

#Y = investmentUSD / powerCapacityMW

#Interpretation:
#Reported project investment per MW of reported project power capacity.

#Eligibility Gate 1:
#- investmentUSD must be present and > 0
#- powerCapacityMW must be present and > 0

#The target is not assumed to represent actual outturn construction CAPEX.

# =====
# BLOCK 3B – DEFINE TARGET AND CANDIDATE FEATURES
# =====

target = "investmentPerMW"

candidate_features = [
    "powerCapacityMW",
    "totalSquareFeet",
    "state",
    "region",
    "numberOfPhases",
    "purpose",
    "renewableEnergy",
    "pueTarget",
    "waterCooling"
]

print("TARGET")
print("-----")
print(target)

print("\nCANDIDATE FEATURES")
print("-----")

for feature in candidate_features:
    print(feature)

```

TARGET

investmentPerMW

CANDIDATE FEATURES

powerCapacityMW

totalSquareFeet

state

region

numberOfPhases

purpose

renewableEnergy

pueTarget

waterCooling

```
In [8]: # =====
# BLOCK 3C – FEATURE AVAILABILITY AUDIT
# =====

feature_audit = pd.DataFrame({
    "dtype": cost_data[candidate_features].dtypes.astype(str),
    "available": cost_data[candidate_features].notna().sum(),
    "missing": cost_data[candidate_features].isna().sum(),
    "missing_pct": (
        cost_data[candidate_features].isna().mean() * 100
    ).round(1),
    "unique_values": cost_data[candidate_features].nunique()
})

feature_audit = feature_audit.sort_values("missing_pct")

display(feature_audit)
```

	dtype	available	missing	missing_pct	unique_values
powerCapacityMW	float64	616	0	0.0	249
state	object	616	0	0.0	49
purpose	object	607	9	1.5	88
totalSquareFeet	float64	508	108	17.5	318
numberOfPhases	float64	447	169	27.4	7
region	object	433	183	29.7	165
waterCooling	object	332	284	46.1	251
renewableEnergy	object	261	355	57.6	74
pueTarget	float64	32	584	94.8	16

```
In [9]: # =====
# BLOCK 3D – CATEGORICAL FEATURE AUDIT
# =====

categorical_features = [
    "state",
    "region",
    "purpose",
    "renewableEnergy",
    "waterCooling"
]
```

```

for feature in categorical_features:

    print("\n" + "=" * 60)
    print(feature.upper())
    print("=" * 60)

    display(
        cost_data[feature]
        .value_counts(dropna=False)
        .head(20)
    )

```

=====

STATE

=====

state	
Texas	44
Virginia	42
Illinois	36
Georgia	34
California	23
Arizona	23
Nevada	23
Ohio	18
Missouri	18
Indiana	17
Alabama	17
New Jersey	17
Utah	16
Michigan	14
South Carolina	14
New York	13
North Dakota	12
North Carolina	12
Pennsylvania	12
Kentucky	12

Name: count, dtype: int64

=====

REGION

=====

region	
NaN	183
Northern Virginia	21
Greater Phoenix	16
Northern New Jersey	14
Chicago Suburbs	12
Silicon Valley	11
Dallas-Fort Worth Metro	10
Salt Lake Metro	9
Northern Nevada	9
Southern Nevada	9
Central Ohio	7
South Metro Atlanta	6
Omaha Metropolitan Area	6
Western New York	5
Western Maryland	5
Central Iowa	5
Chicago	5
Western Chicago Suburbs	5
Central Washington	5
Southwestern Idaho	5

Name: count, dtype: int64

=====

PURPOSE

=====

purpose	
hyperscale	72
colocation	56
hyperscale; cloud	49
other	37
hyperscale; ai-ml	35
hyperscale; colocation	32
ai-ml; hyperscale	25
colocation; enterprise	25
colocation; hyperscale	22
ai-ml	21
cloud; hyperscale	21
ai-ml; cloud; hyperscale	20
colocation; cloud	15
hyperscale; cloud; ai-ml	12
hyperscale; ai-ml; cloud	10
ai-ml; cloud	10
colocation; ai-ml	9
NaN	9
enterprise	8
cloud	6

Name: count, dtype: int64

=====

RENEWABLEENERGY

=====

renewableEnergy
NaN
355
TRUE
107
FALSE
82
Co-located with more than 1 GW of wind, solar, and battery storage systems; most site power expected from clean energy on day one, with a minority met by on-site gas firming for reliability. 1
100% renewable energy source
1
New solar generation and battery storage (reported).
1
Matched with 100% renewable energy (reported; linked to TVA partnership adding 530 MW of renewable energy in Tennessee)
1
5C said it would source at least 30% of its power from wind, solar, and/or nuclear resources through power purchase agreements with Ohio-based renewable-energy developers. 1
QTS pages state clean-energy goals, but no project-specific PPA, renewable-energy percentage, or on-site generation plan was located.
1
Access to renewable energy (details not specified)
1
On-site solar PV and fuel cells; procurement of RECs via NC GreenPower for remaining demand
1
100% renewable-energy coverage
1
DataBank markets SLC6 with a 100% renewable-energy source claim.
1
100% renewable-energy coverage; source does not specify on-site generation, retail supply, PPAs, or unbundled certificates
1
100% renewable-energy coverage through U.S. wind VPPA RECs and Green-e wind RECs; includes a 450 kW solar photovoltaic system.
1
50-acre 14 MW solar field; historical materials describe a 14.1 MW solar system for McGraw-Hill's East Windsor offices and data center.
1
10-year renewable-energy agreement for renewable energy equal to expected electricity needs of Chicago and Piscataway data centers; current campus also lists renewable-sourced power and a 2 MW rooftop solar field. 1
Renewable energy partnership with NV Energy announced; no public source confirms a specific PPA or 100% renewable match.
1
Switch Las Vegas campus advertises 100% renewable energy; no Cxtera/Centersquare LAS12-specific power-purchase agreement was located.
1
Oncor deregulated power territory with access to retail suppliers and renewable programs
1
Name: count, dtype: int64
=====

WATERCOOLING

=====

```

waterCooling
NaN
284
FALSE
41
TRUE
37
closed-loop cooling system that does not consume water for cooling once operationa
1
2
air-cooled
2
closed-loop cooling system
2
water-free cooling
2
closed-loop cooling
2
AWS reported using outside air for most of the year and water for cooling about 7%
of the year in Oregon; Hermiston water strategy uses aquifer storage and recovery
for seasonal water supply.
1
industrial cooling-water discharge; onsite wastewater pond storage proposed for PD
X 154 until Port of Morrow off-site discharge infrastructure is available
1
air-cooled chillers and closed-loop water system
1
Delta3 air-cooling arrays with secondary loops for DeltaFlow liquid cooling and cl
osed-loop chilled-water system; air plus closed-loop water/glycol recirculation
1
closed-loop; direct liquid cooling using recirculating fluid
1
indirect air-side economizer cooling
1
Liquid-ready high-density cooling capabilities; Flexential company-level design st
atement says 4th- and 5th-generation data centers are designed with WUE of zero, l
ow PUEs in the 1.25-1.40 range, Energy Star efficiency-rated UPS systems, and Next
Gen SuperCRAC cooling.      1
closed-loop cooling systems; COL7 also lists air-cooled chillers and liquid-to-chi
p readiness, and COL8 lists liquid-cooling readiness with configurable 70-90% liqu
id-cooled mix
1
Free cooling much of the year, energy-efficient HVAC, and liquid-cooling compatibi
lity for high-density deployments; no quantified water design found.
1
Air-based cooling for Phase 1; original 2023 concept referenced recycled water for
cooling.
1
full economization mode
1
evaporative cooling during peak thermal conditions
1
Name: count, dtype: int64

```

```

In [10]: # =====
# BLOCK 3E - NUMERICAL FEATURE AUDIT
# =====

numeric_features = [
    "powerCapacityMW",
    "totalSquareFeet",
    "numberOfPhases",
    "pueTarget"
]

```

```
display(
    cost_data[numeric_features].describe().T
)
```

	count	mean	std	min	25%	50%	75%
powerCapacityMW	616.0	4.122895e+02	8.965202e+02	0.16	31.935	150.000	4.447500e+02
totalSquareFeet	508.0	1.993982e+06	1.071372e+07	0.00	155557.500	575445.000	1.515000e+06
numberOfPhases	447.0	4.832215e-01	9.928265e-01	0.00	0.000	0.000	0.000000e+00
pueTarget	32.0	1.295313e+00	2.048319e-01	1.08	1.150	1.275	1.362500e+00

```
In [11]: # =====
# BLOCK 3G - GLOBAL OUTSIDE-VIEW BASELINE
# =====

global_median = cost_data["investmentPerMW"].median()
global_mean = cost_data["investmentPerMW"].mean()

print("GLOBAL OUTSIDE VIEW")
print("-----")

print(
    f"Mean capital intensity: "
    f"${global_mean / 1_000_000:,.2f}m/MW"
)

print(
    f"Median capital intensity: "
    f"${global_median / 1_000_000:,.2f}m/MW"
)

GLOBAL OUTSIDE VIEW
-----
Mean capital intensity:  $10.49m/MW
Median capital intensity: $7.00m/MW
```

```
In [12]: # =====
# FIRST NAIVE ATLAS ESTIMATE
# =====

target_capacity = 100

estimated_investment = (
    target_capacity * global_median
)

print(f"Target capacity: {target_capacity} MW")

print(
    f"Estimated investment: "
    f"${estimated_investment / 1_000_000:,.1f}m"
)

Target capacity: 100 MW
Estimated investment: $700.0m
```

```
In [13]: # =====
# BLOCK 4B - BUILD FIRST ML DATASET
```

```
# =====

from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestNeighbors

ml_features = [
    "powerCapacityMW",
    "totalSquareFeet"
]

ml_data = cost_data[
    cost_data[ml_features].notna().all(axis=1)
].copy()

print("ML DATASET")
print("-----")
print(f"RCF eligible projects: {len(cost_data):,}")
print(f"Projects with required ML features: {len(ml_data):,}")
print(
    f"Coverage: "
    f"{len(ml_data) / len(cost_data) * 100:.1f}%"
)

display(
    ml_data[
        [
            "projectName",
            "powerCapacityMW",
            "totalSquareFeet",
            "investmentPerMW"
        ]
    ].head()
)
```

ML DATASET

RCF eligible projects: 616

Projects with required ML features: 508

Coverage: 82.5%

	projectName	powerCapacityMW	totalSquareFeet	investmentPerMW
0	Meta Montgomery Data Center	195.0	1300000.0	7.692308e+06
1	Meta Huntsville Data Center	214.0	3500000.0	7.009346e+06
2	Google Bridgeport Data Center	143.0	270000.0	1.398601e+07
3	Project Marvel - Bessemer Hyperscale Data Center	1200.0	4500000.0	1.208333e+07
5	DC BLOX Huntsville Expansion	5.0	38000.0	7.000000e+06

```
In [14]: # =====
# BLOCK 4C - DEFINE X AND Y
# =====

X = ml_data[ml_features]

y = ml_data["investmentPerMW"]

print("X shape:", X.shape)
print("y shape:", y.shape)

display(X.head())
```

X shape: (508, 2)
y shape: (508,)

	powerCapacityMW	totalSquareFeet
0	195.0	1300000.0
1	214.0	3500000.0
2	143.0	270000.0
3	1200.0	4500000.0
5	5.0	38000.0

```
In [15]: # =====
# BLOCK 4D - STANDARDISE FEATURES
# =====

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

print("Original first 5 projects:")
display(X.head())

print("\nStandardised first 5 projects:")
display(
    pd.DataFrame(
        X_scaled,
        columns=ml_features
    ).head()
)
```

Original first 5 projects:

	powerCapacityMW	totalSquareFeet
0	195.0	1300000.0
1	214.0	3500000.0
2	143.0	270000.0
3	1200.0	4500000.0
5	5.0	38000.0

Standardised first 5 projects:

	powerCapacityMW	totalSquareFeet
0	-0.230464	-0.064839
1	-0.203161	0.140708
2	-0.305188	-0.161072
3	1.213708	0.234138
4	-0.503492	-0.182748

```
In [16]: # =====
# BLOCK 4E - BUILD kNN REFERENCE-CLASS SELECTOR
# =====

k = 10
```

```

knn = NearestNeighbors(
    n_neighbors=k,
    metric="euclidean"
)

knn.fit(X_scaled)

print("kNN reference-class selector fitted.")
print(f"k = {k}")

```

kNN reference-class selector fitted.
k = 10

```

In [18]: # =====
# BLOCK 4F – DEFINE TARGET PROJECT
# =====
#Testing a sample project
target_project = pd.DataFrame({
    "powerCapacityMW": [150],
    "totalSquareFeet": [1_000_000]
})

display(target_project)
target_scaled = scaler.transform(target_project)

display(
    pd.DataFrame(
        target_scaled,
        columns=ml_features
    )
)

```

	powerCapacityMW	totalSquareFeet
0	150	1000000

	powerCapacityMW	totalSquareFeet
0	-0.295129	-0.092868

```

In [19]: # =====
# BLOCK 4G – FIND NEAREST HISTORICAL PROJECTS
# =====

distances, indices = knn.kneighbors(target_scaled)

reference_class = ml_data.iloc[
    indices[0]
].copy()

reference_class["distance"] = distances[0]

reference_class = reference_class.sort_values("distance")

display(
    reference_class[
        [
            "projectName",
            "state",
            "region",
            "powerCapacityMW",
            "totalSquareFeet",
            "investmentUSD",

```

```
        "investmentPerMW",
        "distance"
    ]
]
)
```

	projectName	state	region	powerCapacityMW	totalSquareFeet	investmentUSD	inve:
119	DC BLOX - Atlanta East (Rockdale County)	Georgia	East Metro Atlanta	144.0	1000000.0	1.200000e+09	
482	Meta - Temple Data Center	Texas	Central Texas	152.0	900000.0	8.000000e+08	
18	Vantage Data Centers Phoenix Campus	Arizona	Greater Phoenix	160.0	1000000.0	1.500000e+09	
500	Cirrus Data Services - View 78 Campus	Utah	Salt Lake Metro	160.0	1000000.0	1.120000e+09	
418	Flexential Hillsboro Data Center Campus	Oregon	Portland Metro	140.3	1200000.0	9.450000e+08	
358	Niagara Digital Campus	New York	Western New York	140.0	1232715.0	1.500000e+09	
254	Meta Rosemount Data Center	Minnesota	Twin Cities Metro	150.0	715000.0	8.000000e+08	
536	Stack Infrastructure Loudoun County Campus	Virginia	Northern Virginia	144.0	1310000.0	1.008000e+09	
494	Novva Data Centers - West Jordan Campus	Utah	Salt Lake Metro	175.0	1000000.0	2.000000e+09	
744	Metrobloks Liberty Data Center Campus	Missouri	NaN	150.0	568800.0	1.400000e+09	



```
In [20]: # =====
# BLOCK 4H - REFERENCE-CLASS COST DISTRIBUTION
# =====

rc_costs = reference_class["investmentPerMW"]

p25 = rc_costs.quantile(0.25)
p50 = rc_costs.quantile(0.50)
```

```

p75 = rc_costs.quantile(0.75)

print("REFERENCE-CLASS CAPITAL INTENSITY")
print("-----")

print(f"P25:    ${p25 / 1_000_000:,.2f}m/MW")
print(f"Median:${p50 / 1_000_000:,.2f}m/MW")
print(f"P75:    ${p75 / 1_000_000:,.2f}m/MW")

```

REFERENCE-CLASS CAPITAL INTENSITY

P25: \$6.80m/MW

Median:\$7.67m/MW

P75: \$9.36m/MW

```

In [21]: # =====
# BLOCK 4I – IMPLIED INVESTMENT ESTIMATE
# =====

target_mw = target_project["powerCapacityMW"].iloc[0]

investment_p25 = target_mw * p25
investment_p50 = target_mw * p50
investment_p75 = target_mw * p75

print("ATLAS RCF ESTIMATE")
print("-----")

print(f"Target capacity: {target_mw:,.0f} MW")

print("\nCapital intensity:")
print(f"P25:    ${p25 / 1_000_000:,.2f}m/MW")
print(f"P50:    ${p50 / 1_000_000:,.2f}m/MW")
print(f"P75:    ${p75 / 1_000_000:,.2f}m/MW")

print("\nImplied reported investment:")
print(f"P25:    ${investment_p25 / 1_000_000:,.1f}m")
print(f"P50:    ${investment_p50 / 1_000_000:,.1f}m")
print(f"P75:    ${investment_p75 / 1_000_000:,.1f}m")

```

ATLAS RCF ESTIMATE

Target capacity: 150 MW

Capital intensity:

P25: \$6.80m/MW

P50: \$7.67m/MW

P75: \$9.36m/MW

Implied reported investment:

P25: \$1,020.3m

P50: \$1,150.0m

P75: \$1,404.7m

```

In [22]: # =====
# BLOCK 4J – GLOBAL VS ML-SELECTED OUTSIDE VIEW
# =====

global_p50 = cost_data["investmentPerMW"].median()

print("GLOBAL OUTSIDE VIEW")
print("-----")
print(
    f"${global_p50 / 1_000_000:,.2f}m/MW"
)

```

```

print("\nML-SELECTED REFERENCE CLASS")
print("-----")
print(
    f"${p50 / 1_000_000:,.2f}m/MW"
)

difference = (
    (p50 / global_p50) - 1
) * 100

print(
    f"\nDifference: {difference:+.1f}%"
)

```

GLOBAL OUTSIDE VIEW

\$7.00m/MW

ML-SELECTED REFERENCE CLASS

\$7.67m/MW

Difference: +9.5%

```

In [30]: #Block 5: ATLAS RCF v0.1
# =====
# BLOCK 5B – ATLAS RCF v0.1 ESTIMATION ENGINE
# =====

from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestNeighbors

def atlas_estimate(power_mw, square_feet, k=10, data=cost_data):

    # Features used to define similarity
    features = [
        "powerCapacityMW",
        "totalSquareFeet"
    ]

    # Validate target project
    if power_mw <= 0:
        raise ValueError("power_mw must be greater than zero.")

    if square_feet <= 0:
        raise ValueError("square_feet must be greater than zero.")

    if k < 3:
        raise ValueError("k must be at least 3.")

    # Build eligible historical universe
    universe = data[
        data[features].notna().all(axis=1)
        & (data["powerCapacityMW"] > 0)
        & (data["totalSquareFeet"] > 0)
        & (data["investmentPerMW"] > 0)
    ].copy()

    if k > len(universe):
        raise ValueError(
            f"k={k} exceeds eligible universe of {len(universe)} projects."
        )

```

```
# Historical feature matrix
X = universe[features]

# Standardise features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Define new target project
target = pd.DataFrame({
    "powerCapacityMW": [power_mw],
    "totalSquareFeet": [square_feet]
})

target_scaled = scaler.transform(target)

# Find nearest neighbours
knn = NearestNeighbors(
    n_neighbors=k,
    metric="euclidean"
)

knn.fit(X_scaled)

distances, indices = knn.kneighbors(target_scaled)

# Construct reference class
reference_class = universe.iloc[indices[0]].copy()
reference_class["distance"] = distances[0]
reference_class = reference_class.sort_values("distance")

# Empirical reference-class distribution
rc_costs = reference_class["investmentPerMW"]

p25 = rc_costs.quantile(0.25)
p50 = rc_costs.quantile(0.50)
p75 = rc_costs.quantile(0.75)

# Global outside-view benchmark
global_median = universe["investmentPerMW"].median()

# Return results
return {
    "target_power_mw": power_mw,
    "target_square_feet": square_feet,
    "eligible_universe": len(universe),
    "k": k,

    "global_median_per_mw": global_median,

    "rc_p25_per_mw": p25,
    "rc_p50_per_mw": p50,
    "rc_p75_per_mw": p75,

    "global_investment": power_mw * global_median,

    "investment_p25": power_mw * p25,
    "investment_p50": power_mw * p50,
    "investment_p75": power_mw * p75,

    "reference_class": reference_class
}
```

```
In [31]: print(atlas_estimate)
```

```
<function atlas_estimate at 0x0000025FAA424360>
```

```
In [32]: result = atlas_estimate(  
    power_mw=150,  
    square_feet=1_000_000,  
    k=10  
)  
  
result
```

```
Out[32]: {'target_power_mw': 150,
'target_square_feet': 1000000,
'eligible_universe': 503,
'k': 10,
'global_median_per_mw': 7000000.0,
'rc_p25_per_mw': 6801674.982181041,
'rc_p50_per_mw': 7666666.666666666,
'rc_p75_per_mw': 9364583.333333334,
'global_investment': 105000000.0,
'investment_p25': 1020251247.3271561,
'investment_p50': 115000000.0,
'investment_p75': 1404687500.0,
'reference_class':
```

		projectName	state
city \			
119	DC BLOX - Atlanta East (Rockdale County)	Georgia	Conyers
482	Meta - Temple Data Center	Texas	Temple
500	Cirrus Data Services - View 78 Campus	Utah	Midvale
18	Vantage Data Centers Phoenix Campus	Arizona	Goodyear
418	Flexential Hillsboro Data Center Campus	Oregon	Hillsboro
358	Niagara Digital Campus	New York	Niagara Falls
254	Meta Rosemount Data Center	Minnesota	Rosemount
536	Stack Infrastructure Loudoun County Campus	Virginia	Ashburn
494	Novva Data Centers - West Jordan Campus	Utah	West Jordan
744	Metrobloks Liberty Data Center Campus	Missouri	Liberty

	county	region	status \
119	Rockdale	East Metro Atlanta	under-construction
482	Bell County	Central Texas	under-construction
500	Salt Lake County	Salt Lake Metro	planned
18	Maricopa County	Greater Phoenix	under-construction
418	Washington County	Portland Metro	expansion
358	Niagara County	Western New York	planned
254	Dakota County	Twin Cities Metro	under-construction
536	Loudoun County	Northern Virginia	announced
494	Salt Lake County	Salt Lake Metro	under-construction
744	Clay County	NaN	planned

	sponsors \
119	DC BLOX
482	Meta
500	Cirrus Data Services; Gardner Company
18	Vantage Data Centers
418	Flexential
358	Niagara Falls Redevelopment (NFR); Urbacon Dat...
254	Meta Platforms Inc.
536	Stack Infrastructure
494	Novva Data Centers; CIM Group
744	Metrobloks

	operators	tenants \
119	DC BLOX	NaN
482	Meta	NaN
500	Cirrus Data Services	NaN
18	Vantage Data Centers	NaN
418	Flexential	CoreWeave; unnamed Hillsboro 4 customers
358	Niagara Falls Redevelopment	NaN
254	Meta Platforms Inc.	Meta Platforms Inc.
536	Stack Infrastructure	NaN
494	Novva Data Centers	Leading global tech company (fully leased)
744	Metrobloks	NaN

	totalSquareFeet ...	economicImpact_jobs	economicImpact_taxRevenue \
119	1000000.0 ...	60.0	5000000.0
482	900000.0 ...	100.0	NaN

500	1000000.0	...	NaN	NaN
18	1000000.0	...	NaN	NaN
418	1200000.0	...	NaN	NaN
358	1232715.0	...	550.0	414900000.0
254	715000.0	...	100.0	1000000.0
536	1310000.0	...	NaN	NaN
494	1000000.0	...	200.0	NaN
744	568800.0	...	NaN	NaN

economicImpact_notes \

119 DRI Part 2 lists estimated local tax revenues ...
 482 1200 construction jobs; 100 permanent jobs
 500 No public job counts, tax-base estimate, annua...
 18 Vantage/City releases cite about 3,000 "constr...
 418 No campus-wide economic impact figures were fo...
 358 MRB Group economic/fiscal impact study (Mar 20...
 254 Public statements anticipate millions in tax r...
 536 No project-specific economic impact study (job...
 494 West Jordan economic development director stat...
 744 NaN

incentives_type incentives_value \

119 Sales Tax Exemption; Property Tax Abatement; P... NaN
 482 Property Tax Abatement; Infrastructure Support 8000000.0
 500 NaN NaN
 18 NaN NaN
 418 Property Tax Abatement; Other NaN
 358 Property Tax Abatement; Pilot; Cba; Other NaN
 254 Property Tax Abatement NaN
 536 Sales Tax Exemption NaN
 494 Sales Tax Exemption NaN
 744 NaN NaN

challenges \

119 Information gaps: local incentive/PILOT docume...
 482 Construction paused on 2022-12-02 and later re...
 500 The principal risk flag is lack of public evid...
 18 Public records reviewed did not surface incent...
 418 Public scrutiny and controversy over Hillsboro...
 358 Zoning/text-amendment process for HEUL; confli...
 254 Formal abatement agreement/clawbacks, City Cou...
 536 Project-specific permitting/parcel identificat...
 494 Limited public records located for zoning/perm...
 744 NaN

notes lastUpdated \

119 Located approximately 24 miles east of downtow... 15/06/2026
 482 \$800M hyperscale facility; construction resume... 05/02/2026
 500 Cirrus Data Services is a subsidiary of Gardne... 15/06/2026
 18 Secured \$735 million refinancing package for t... 15/06/2026
 418 Flexential announced on 2026-05-28 that it clo... 15/06/2026
 358 140+ acre site downtown Niagara Falls; 5,600 c... 15/06/2026
 254 Meta's 19th data center in the U.S. and 23rd i... 15/06/2026
 536 New 144MW campus in greater Ashburn area; Inve... 15/06/2026
 494 \$2 billion financing secured from J.P. Morgan ... 19/07/2026
 744 Metrobloks announced a high-density, AI-ready ... 15/06/2026

investmentPerMW distance

119 8.333333e+06 0.009036
 482 5.263158e+06 0.009774
 500 7.000000e+06 0.015061
 18 9.375000e+06 0.015061
 418 6.735567e+06 0.023649
 358 1.071429e+07 0.026364

```

254      5.333333e+06  0.026501
536      7.000000e+06  0.030209
494      1.142857e+07  0.037652
744      9.333333e+06  0.040095

```

```
[10 rows x 35 columns]}
```

In [33]: atlas_report(result)

```

-----
NameError                                Traceback (most recent call last)
Cell In[33], line 1
----> 1 atlas_report(result)

NameError: name 'atlas_report' is not defined

```

```

In [34]: # =====
# BLOCK 5C-5G
# ATLAS REPORTING + BACKTEST ENGINE
# =====

import numpy as np
import pandas as pd

from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestNeighbors
from sklearn.metrics import mean_absolute_error, mean_squared_error

# =====
# PART 1 – ATLAS REPORTING LAYER
# =====

def atlas_report(result):

    print("=" * 60)
    print("ATLAS RCF v0.1")
    print("Machine-Learned Reference-Class Cost Estimate")
    print("=" * 60)

    print("\nTARGET PROJECT")
    print("-" * 60)
    print(f"Power capacity:           {result['target_power_mw'],.0f} MW")
    print(f"Gross floor area:         {result['target_square_feet'],.0f} ft²")

    print("\nREFERENCE CLASS")
    print("-" * 60)
    print(f"Eligible universe:         {result['eligible_universe'],} projects")
    print(f"Selected comparables:     {result['k']} projects")
    print("Selection method:         k-nearest neighbours")
    print("Distance metric:          standardised Euclidean")

    print("\nCAPITAL INTENSITY")
    print("-" * 60)
    print(
        f"Global median:           "
        f"${result['global_median_per_mw']/1e6:.2f}m/MW"
    )
    print(
        f"Reference class P25:     "
        f"${result['rc_p25_per_mw']/1e6:.2f}m/MW"
    )
    print(
        f"Reference class P50:     "

```

```

        f"${result['rc_p50_per_mw']/1e6:,.2f}m/MW"
    )
    print(
        f"Reference class P75:      "
        f"${result['rc_p75_per_mw']/1e6:,.2f}m/MW"
    )

    print("\nIMPLIED REPORTED INVESTMENT")
    print("-" * 60)
    print(
        f"Global median:              "
        f"${result['global_investment']/1e9:,.3f}bn"
    )
    print(
        f"P25:                        "
        f"${result['investment_p25']/1e9:,.3f}bn"
    )
    print(
        f"P50:                        "
        f"${result['investment_p50']/1e9:,.3f}bn"
    )
    print(
        f"P75:                        "
        f"${result['investment_p75']/1e9:,.3f}bn"
    )

    print("\nIMPORTANT")
    print("-" * 60)
    print(
        "Estimates are based on reported historical investment."
    )
    print(
        "P25/P50/P75 are empirical reference-class quantiles, "
        "not validated confidence intervals."
    )

# =====
# PART 2 – ATLAS BACKTEST ENGINE
# =====

def atlas_backtest(k=10, data=cost_data):

    features = [
        "powerCapacityMW",
        "totalSquareFeet"
    ]

    # -----
    # 1. BUILD ELIGIBLE UNIVERSE
    # -----

    universe = data[
        data[features].notna().all(axis=1)
        & (data["powerCapacityMW"] > 0)
        & (data["totalSquareFeet"] > 0)
        & (data["investmentPerMW"] > 0)
    ].copy()

    if k < 3:
        raise ValueError("k must be at least 3.")

    if k >= len(universe):
        raise ValueError(

```

```

        f"k={k} must be smaller than the eligible "
        f"universe of {len(universe)} projects."
    )

# -----
# 2. DEFINE X AND Y
# -----

X = universe[features]

y = universe[
    "investmentPerMw"
].to_numpy()

# -----
# 3. STANDARDISE FEATURES
# -----

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

# -----
# 4. FIND k+1 NEIGHBOURS
#
# A project's closest neighbour is itself.
# We therefore request k+1 and remove itself.
# -----

knn = NearestNeighbors(
    n_neighbors=k + 1,
    metric="euclidean"
)

knn.fit(X_scaled)

distances, indices = knn.kneighbors(
    X_scaled
)

# -----
# 5. GENERATE RCF PREDICTIONS
# -----

rcf_predictions = []

for i in range(len(universe)):
    neighbour_indices = [
        idx
        for idx in indices[i]
        if idx != i
   ][:k]

    neighbour_costs = y[
        neighbour_indices
    ]

    prediction = np.median(
        neighbour_costs
    )

    rcf_predictions.append(
        prediction

```

```

    )

rcf_predictions = np.array(
    rcf_predictions
)

# -----
# 6. GENERATE GLOBAL OUTSIDE-VIEW PREDICTIONS
#
# Each project's own target value is excluded.
# -----

global_predictions = []

for i in range(len(y)):

    other_y = np.delete(
        y,
        i
    )

    global_prediction = np.median(
        other_y
    )

    global_predictions.append(
        global_prediction
    )

global_predictions = np.array(
    global_predictions
)

# -----
# 7. CALCULATE PERFORMANCE
# -----

rcf_mae = mean_absolute_error(
    y,
    rcf_predictions
)

global_mae = mean_absolute_error(
    y,
    global_predictions
)

rcf_rmse = np.sqrt(
    mean_squared_error(
        y,
        rcf_predictions
    )
)

global_rmse = np.sqrt(
    mean_squared_error(
        y,
        global_predictions
    )
)

# Positive number = RCF has Lower MAE
mae_change = (
    (global_mae - rcf_mae)

```

```

    / global_mae
) * 100

# -----
# 8. BUILD PROJECT-LEVEL RESULTS TABLE
# -----

results = universe[
    [
        "projectName",
        "powerCapacityMW",
        "totalSquareFeet",
        "investmentPerMW"
    ]
].copy()

results["actual"] = y

results["rcf_prediction"] = (
    rcf_predictions
)

results["global_prediction"] = (
    global_predictions
)

results["rcf_absolute_error"] = np.abs(
    y - rcf_predictions
)

results["global_absolute_error"] = np.abs(
    y - global_predictions
)

# -----
# 9. RETURN EVERYTHING
# -----

return {
    "n": len(universe),
    "k": k,

    "global_mae": global_mae,
    "global_rmse": global_rmse,

    "rcf_mae": rcf_mae,
    "rcf_rmse": rcf_rmse,

    "mae_change": mae_change,

    "results": results
}

# =====
# PART 3 – BACKTEST REPORTING LAYER
# =====

def atlas_backtest_report(backtest):

    print("=" * 60)
    print("ATLAS RCF v0.1 – HISTORICAL BACKTEST")
    print("=" * 60)

```

```

print("\nTEST")
print("-" * 60)
print(f"Projects tested:      {backtest['n'],}")
print(f"k:                    {backtest['k'],}")

print("\nGLOBAL OUTSIDE VIEW")
print("-" * 60)
print(
    f"MAE:                    "
    f"${backtest['global_mae']/1e6:,.2f}m/MW"
)
print(
    f"RMSE:                    "
    f"${backtest['global_rmse']/1e6:,.2f}m/MW"
)

print("\nML-SELECTED REFERENCE CLASS")
print("-" * 60)
print(
    f"MAE:                    "
    f"${backtest['rcf_mae']/1e6:,.2f}m/MW"
)
print(
    f"RMSE:                    "
    f"${backtest['rcf_rmse']/1e6:,.2f}m/MW"
)

print("\nCOMPARISON")
print("-" * 60)
print(
    f"MAE change vs global:  "
    f"${backtest['mae_change']:+.1f}%"
)

print("\nINTERPRETATION")
print("-" * 60)

if backtest["mae_change"] > 0:
    print(
        "RCF produced lower MAE than the global baseline "
        "in this preliminary historical backtest."
    )

elif backtest["mae_change"] < 0:
    print(
        "RCF produced higher MAE than the global baseline "
        "in this preliminary historical backtest."
    )

else:
    print(
        "RCF and the global baseline produced the same MAE "
        "in this preliminary historical backtest."
    )

print(
    "\nThis is not yet final out-of-sample validation."
)

```

```

In [35]: # =====
# RUN ATLAS ESTIMATE
# =====

result = atlas_estimate(

```

```

power_mw=150,
square_feet=1_000_000,
k=10
)

atlas_report(result)

```

```

=====
ATLAS RCF v0.1
Machine-Learned Reference-Class Cost Estimate
=====

TARGET PROJECT
-----
Power capacity:      150 MW
Gross floor area:    1,000,000 ft²

REFERENCE CLASS
-----
Eligible universe:   503 projects
Selected comparables: 10 projects
Selection method:    k-nearest neighbours
Distance metric:     standardised Euclidean

CAPITAL INTENSITY
-----
Global median:       $7.00m/MW
Reference class P25:  $6.80m/MW
Reference class P50:  $7.67m/MW
Reference class P75:  $9.36m/MW

IMPLIED REPORTED INVESTMENT
-----
Global median:       $1.050bn
P25:                 $1.020bn
P50:                 $1.150bn
P75:                 $1.405bn

IMPORTANT
-----
Estimates are based on reported historical investment.
P25/P50/P75 are empirical reference-class quantiles, not validated confidence inte
rvals.

```

```

In [36]: # =====
# RUN ATLAS BACKTEST
# =====

backtest = atlas_backtest(
    k=10
)

atlas_backtest_report(backtest)

```

```
=====
ATLAS RCF v0.1 – HISTORICAL BACKTEST
=====
```

```
TEST
-----
```

```
Projects tested:    503
k:                 10
```

```
GLOBAL OUTSIDE VIEW
-----
```

```
MAE:               $6.00m/MW
RMSE:              $14.60m/MW
```

```
ML-SELECTED REFERENCE CLASS
-----
```

```
MAE:               $6.34m/MW
RMSE:              $14.55m/MW
```

```
COMPARISON
-----
```

```
MAE change vs global: -5.6%
```

```
INTERPRETATION
-----
```

RCF produced higher MAE than the global baseline in this preliminary historical backtest.

This is not yet final out-of-sample validation.

```
In [37]: # =====
# INSPECT 5 LARGEST RCF ERRORS
# =====

worst_5 = (
    backtest["results"]
    .sort_values(
        "rcf_absolute_error",
        ascending=False
    )
    .head(5)
)

display(
    worst_5[
        [
            "projectName",
            "powerCapacityMW",
            "totalSquareFeet",
            "actual",
            "rcf_prediction",
            "global_prediction",
            "rcf_absolute_error"
        ]
    ]
)
```

	projectName	powerCapacityMW	totalSquareFeet	actual	rcf_prediction	global_predic
6	DC BLOX Birmingham Data Center	5.0	31000.0	1.570000e+08	7.000000e+06	70000
636	DC BLOX Birmingham Expansion	5.0	36500.0	1.570000e+08	1.050000e+07	70000
631	Nebius Birmingham AI Factory	300.0	500000.0	1.196667e+08	6.175926e+06	70000
40	Conway Data Center (Forgelight Ventures)	10.0	300000.0	1.000000e+08	7.944444e+06	70000
201	Eaton Corporation - Blue Data Center	1.0	102500.0	8.000000e+07	8.500000e+06	70000



```
In [38]: atlas_backtest_report
```

```
Out[38]: <function __main__.atlas_backtest_report(backtest)>
```

```
In [39]: # =====
# BLOCK 5J - ERROR / TARGET DIAGNOSTIC
# =====

diagnostic = backtest["results"].copy()

diagnostic["actual_m_per_mw"] = (
    diagnostic["actual"] / 1_000_000
)

diagnostic["rcf_pred_m_per_mw"] = (
    diagnostic["rcf_prediction"] / 1_000_000
)

diagnostic["error_m_per_mw"] = (
    diagnostic["rcf_absolute_error"] / 1_000_000
)

print("CAPITAL INTENSITY DISTRIBUTION ($m/MW)")
print("-" * 60)

print(
    diagnostic["actual_m_per_mw"]
    .describe(
        percentiles=[
            .50, .75, .90, .95, .99
        ]
    )
)

print("\nCAPACITY OF HIGHEST-INTENSITY PROJECTS")
print("-" * 60)
```

```
display(
  diagnostic
  .sort_values(
    "actual",
    ascending=False
  )
  [
    [
      "projectName",
      "powerCapacityMW",
      "totalSquareFeet",
      "actual_m_per_mw",
      "rcf_pred_m_per_mw",
      "error_m_per_mw"
    ]
  ]
  .head(20)
)
```

CAPITAL INTENSITY DISTRIBUTION (\$m/MW)

count	503.000000
mean	10.293706
std	14.233583
min	0.032000
50%	7.000000
75%	11.111111
90%	20.000000
95%	25.964103
99%	59.972932
max	157.000000
Name: actual_m_per_mw, dtype: float64	

CAPACITY OF HIGHEST-INTENSITY PROJECTS

	projectName	powerCapacityMW	totalSquareFeet	actual_m_per_mw	rcf_pred_m_per_mw	er
636	DC BLOX Birmingham Expansion	5.00	36500.0	157.000000	10.500000	
6	DC BLOX Birmingham Data Center	5.00	31000.0	157.000000	7.000000	
631	Nebius Birmingham AI Factory	300.00	500000.0	119.666667	6.175926	
40	Conway Data Center (Forgelight Ventures)	10.00	300000.0	100.000000	7.944444	
201	Eaton Corporation - Blue Data Center	1.00	102500.0	80.000000	8.500000	
258	Tract Cannon Falls Technology Park	500.00	1750000.0	60.000000	8.296296	
412	Amazon AWS US West (Oregon) Region - Boardman	266.00	5632000.0	58.646617	7.002615	
270	AVAIO Digital Taurus Data Center Hub	116.00	600000.0	51.724138	7.017544	
550	Port of Walla Walla - Project Forest	100.00	3400000.0	48.000000	7.254673	
97	365 Data Centers Tampa Expansion	0.75	27300.0	46.666667	8.477011	
1001	Boxminer La Pine Data Center	20.00	20000.0	40.000000	9.222222	
357	BlackRock Amherst Data Center	2.00	31000.0	39.900000	7.000000	
355	Stream Data Centers STAMP Campus	500.00	900000.0	38.920000	6.730124	
174	Digital Crossroads Hammond Expansion	180.00	450000.0	38.888889	8.157895	
531	QTS Ashburn IAD2	6.00	87000.0	37.333333	6.596154	

	projectName	powerCapacityMW	totalSquareFeet	actual_m_per_mw	rcf_pred_m_per_mw	er
449	DartPoints Columbia Data Center	2.50	15000.0	35.000000	7.071429	
187	Apple - Wauke Data Center Campus	40.00	400000.0	32.500000	8.750000	
269	Compass Datacenters Meridian Campus	320.00	2000000.0	31.250000	8.500000	
647	Imperial Valley Data Center Campus	330.00	950000.0	30.303030	6.175926	
207	Hut 8 West Feliciano AI	330.00	900000.0	30.303030	6.175926	

```
In [40]: diagnostic["capacity_band"] = pd.cut(
    diagnostic["powerCapacityMW"],
    bins=[
        0,
        10,
        50,
        100,
        250,
        500,
        float("inf")
    ],
    labels=[
        "0-10 MW",
        "10-50 MW",
        "50-100 MW",
        "100-250 MW",
        "250-500 MW",
        "500+ MW"
    ]
)

capacity_diagnostic = (
    diagnostic
    .groupby(
        "capacity_band",
        observed=True
    )
    .agg(
        projects=("actual", "size"),
        median_cost_per_mw=("actual_m_per_mw", "median"),
        mean_cost_per_mw=("actual_m_per_mw", "mean"),
        median_rcf_error=("error_m_per_mw", "median"),
        mean_rcf_error=("error_m_per_mw", "mean")
    )
)

display(capacity_diagnostic)
```

	projects	median_cost_per_mw	mean_cost_per_mw	median_rcf_error	mean_rcf_error
capacity_band					
0-10 MW	80	7.171776	16.295504	3.633942	11.218631
10-50 MW	98	7.000000	8.967267	3.110085	4.701591
50-100 MW	60	7.000000	8.488624	2.303310	4.316016
100-250 MW	98	7.000000	8.843595	2.484411	4.317309
250-500 MW	75	7.000000	12.543852	4.000000	8.902186
500+ MW	92	6.458333	7.375251	4.768001	5.238927

```
In [41]: print("ATLAS BLOCK 5 FINAL RESULT")
print("=" * 50)

print(f"Eligible universe: {backtest['n']}")
print(f"k: {backtest['k']}")

print("\nGLOBAL")
print(f"MAE:  ${backtest['global_mae']/1e6:,.2f}m/MW")
print(f"RMSE: ${backtest['global_rmse']/1e6:,.2f}m/MW")

print("\nRCF")
print(f"MAE:  ${backtest['rcf_mae']/1e6:,.2f}m/MW")
print(f"RMSE: ${backtest['rcf_rmse']/1e6:,.2f}m/MW")

print(
    f"\nMAE change: "
    f"{backtest['mae_change']:+.1f}%"
)
```

ATLAS BLOCK 5 FINAL RESULT
=====

Eligible universe: 503
k: 10

GLOBAL
MAE: \$6.00m/MW
RMSE: \$14.60m/MW

RCF
MAE: \$6.34m/MW
RMSE: \$14.55m/MW

MAE change: -5.6%

```
In [42]: # =====
# BLOCK 6A – TARGET MEASUREMENT AUDIT
# =====

audit_columns = [
    "projectName",
    "status",
    "powerCapacityMW",
    "itLoadMW",
    "investmentUSD",
    "investmentPerMW",
    "totalSquareFeet",
]
```

```
    "numberOfPhases",  
    "purpose",  
    "sizeNotes",  
    "announcedDate",  
    "constructionStartDate",  
    "expectedCompletionDate",  
    "actualCompletionDate",  
    "notes"  
]  
  
target_audit = (  
    cost_data[audit_columns]  
    .sort_values(  
        "investmentPerMw",  
        ascending=False  
    )  
    .head(20)  
)  
  
display(target_audit)
```

	projectName	status	powerCapacityMW	itLoadMW	investmentUSD	investmentPerMV
859	Stack Infrastructure Berry Hill Megasite	planned	299.00	NaN	1.000000e+11	3.344482e+0.
6	DC BLOX Birmingham Data Center	operational	5.00	49.000	7.850000e+08	1.570000e+0.
636	DC BLOX Birmingham Expansion	expansion	5.00	NaN	7.850000e+08	1.570000e+0.
631	Nebius Birmingham AI Factory	under-construction	300.00	NaN	3.590000e+10	1.196667e+0.
40	Conway Data Center (Forgelight Ventures)	planned	10.00	NaN	1.000000e+09	1.000000e+0.
201	Eaton Corporation - Blue Data Center	operational	1.00	1.000	8.000000e+07	8.000000e+0
258	Tract Cannon Falls Technology Park	planned	500.00	NaN	3.000000e+10	6.000000e+0
412	Amazon AWS US West (Oregon) Region - Boardman	expansion	266.00	NaN	1.560000e+10	5.864662e+0
270	AVAIO Digital Taurus Data Center Hub	under-construction	116.00	NaN	6.000000e+09	5.172414e+0
550	Port of Walla Walla - Project Forest	planned	100.00	NaN	4.800000e+09	4.800000e+0
97	365 Data Centers Tampa Expansion	operational	0.75	NaN	3.500000e+07	4.666667e+0
1008	Keel Sharon Data Center	expansion	110.00	NaN	5.000000e+09	4.545455e+0

	projectName	status	powerCapacityMW	itLoadMW	investmentUSD	investmentPerMV
	Expansion					
1001	Boxminer La Pine Data Center	canceled	20.00	NaN	8.000000e+08	4.000000e+0
357	BlackRock Amherst Data Center	operational	2.00	NaN	7.980000e+07	3.990000e+0
355	Stream Data Centers STAMP Campus	planned	500.00	NaN	1.946000e+10	3.892000e+0
174	Digital Crossroads Hammond Expansion	canceled	180.00	NaN	7.000000e+09	3.888889e+0
531	QTS Ashburn IAD2	operational	6.00	NaN	2.240000e+08	3.733333e+0
837	Crusoe Energy Nolan County Data Center	planned	200.00	NaN	7.000000e+09	3.500000e+0
449	DartPoints Columbia Data Center	expansion	2.50	0.905	8.750000e+07	3.500000e+0
19	Vermaland La Osa Data Center Park	planned	1000.00	NaN	3.300000e+10	3.300000e+0

```
In [43]: # =====
# BLOCK 6B - RATIO TARGET DIAGNOSTIC
# =====

ratio_diagnostic = cost_data[
    (cost_data["investmentUSD"] > 0)
    & (cost_data["powerCapacityMW"] > 0)
].copy()

ratio_diagnostic["log_investment"] = np.log(
    ratio_diagnostic["investmentUSD"]
)

ratio_diagnostic["log_power"] = np.log(
```

```

    ratio_diagnostic["powerCapacityMW"]
)

print("CORRELATIONS")
print("-" * 50)

print(
    "Power vs investment/MW:",
    ratio_diagnostic[
        ["powerCapacityMW", "investmentPerMW"]
    ].corr(method="spearman").iloc[0, 1]
)

print(
    "Power vs investment:",
    ratio_diagnostic[
        ["powerCapacityMW", "investmentUSD"]
    ].corr(method="spearman").iloc[0, 1]
)

```

CORRELATIONS

```

-----
Power vs investment/MW: -0.10075765267650592
Power vs investment: 0.8643975520657535

```

```

In [44]: # =====
# BLOCK 6C – SCALE VS INVESTMENT
# =====

import matplotlib.pyplot as plt

plt.figure(figsize=(8, 6))

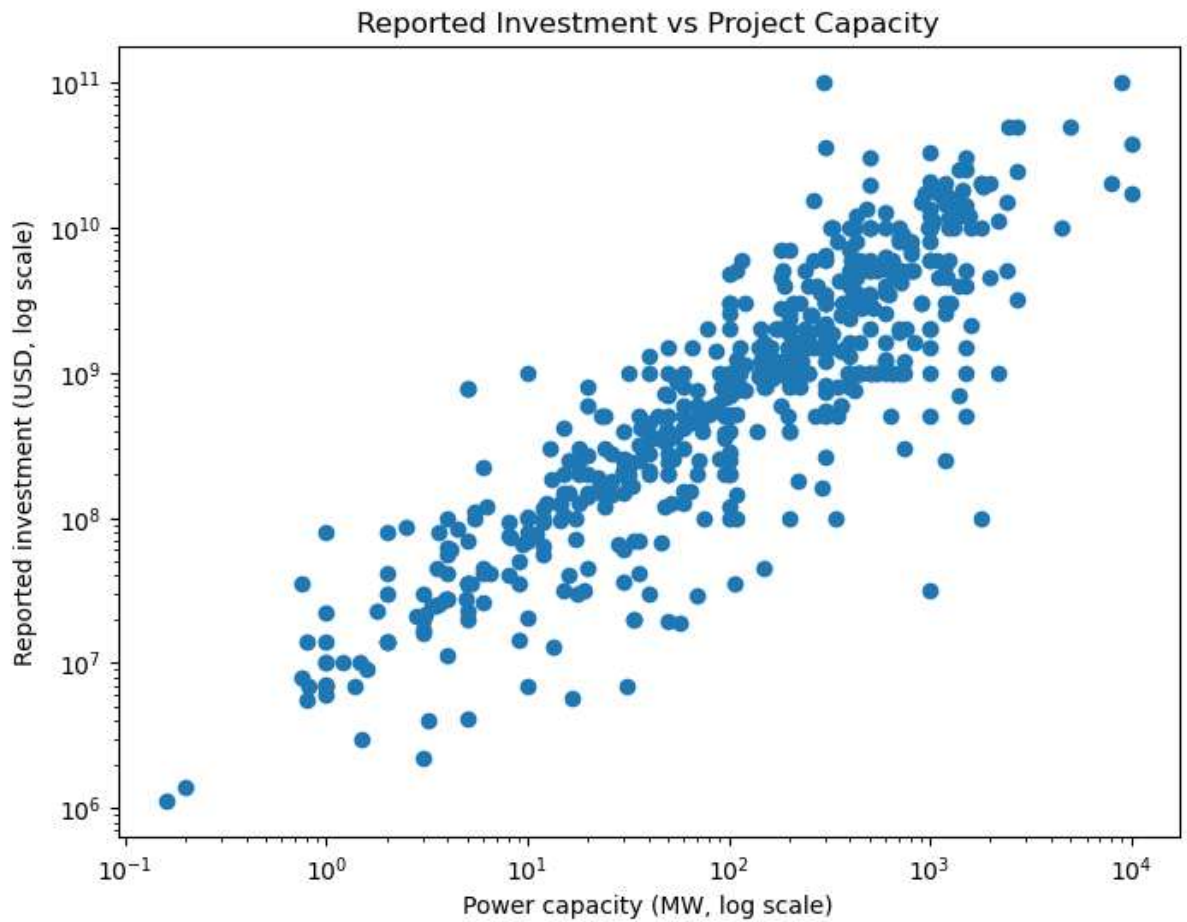
plt.scatter(
    ratio_diagnostic["powerCapacityMW"],
    ratio_diagnostic["investmentUSD"]
)

plt.xscale("log")
plt.yscale("log")

plt.xlabel("Power capacity (MW, log scale)")
plt.ylabel("Reported investment (USD, log scale)")
plt.title("Reported Investment vs Project Capacity")

plt.show()

```



```
In [45]: # =====
# BLOCK 6C.2 – SCALE VS CAPITAL INTENSITY
# =====

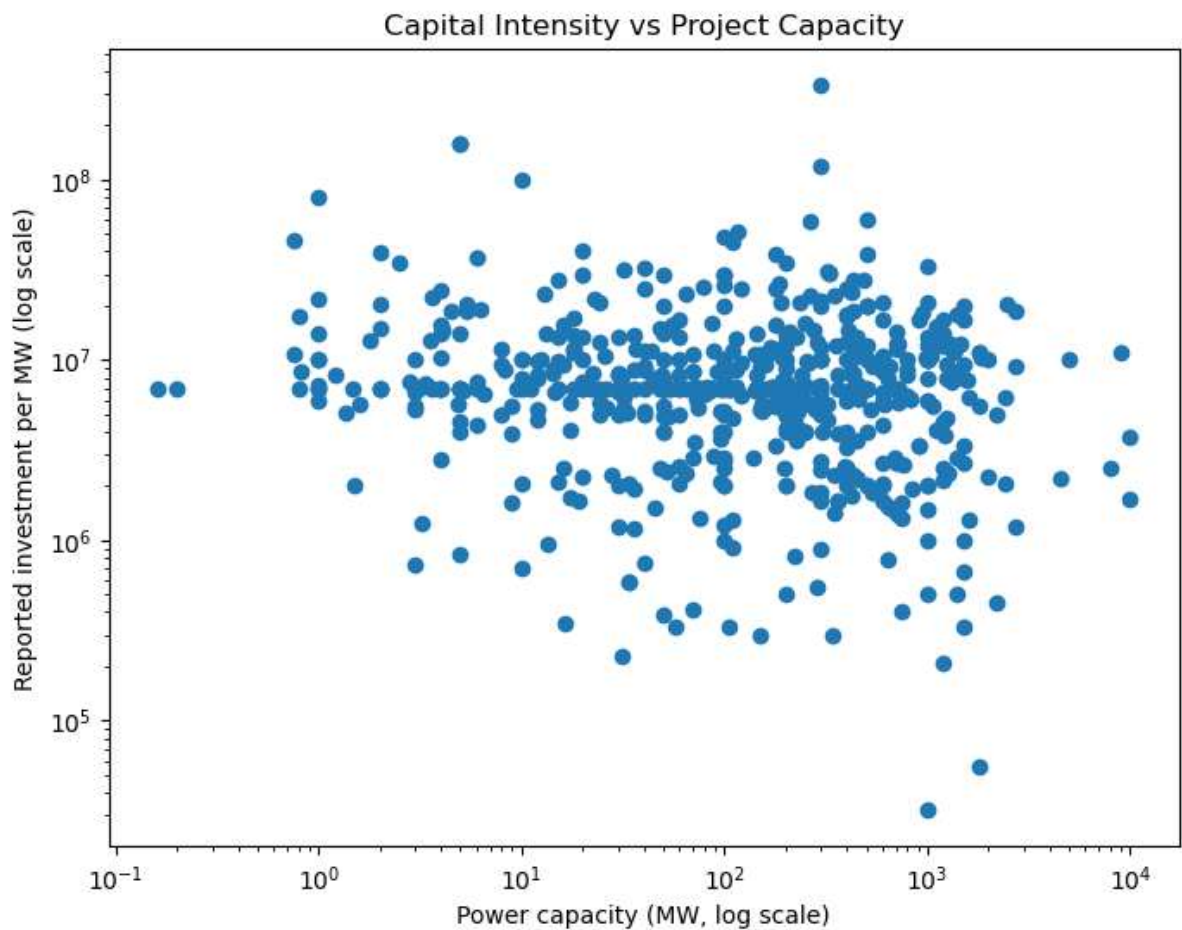
plt.figure(figsize=(8, 6))

plt.scatter(
    ratio_diagnostic["powerCapacityMW"],
    ratio_diagnostic["investmentPerMW"]
)

plt.xscale("log")
plt.yscale("log")

plt.xlabel("Power capacity (MW, log scale)")
plt.ylabel("Reported investment per MW (log scale)")
plt.title("Capital Intensity vs Project Capacity")

plt.show()
```



```
In [46]: # =====
# BLOCK 6D – ESTIMATE SCALE RELATIONSHIP
# =====

from sklearn.linear_model import LinearRegression

scale_model = LinearRegression()

X_scale = ratio_diagnostic[
    "log_power"
]

y_scale = ratio_diagnostic[
    "log_investment"
]

scale_model.fit(
    X_scale,
    y_scale
)

alpha = scale_model.intercept_
beta = scale_model.coef_[0]

r_squared = scale_model.score(
    X_scale,
    y_scale
)

print("SCALING MODEL")
print("=" * 50)

print(f"alpha = {alpha:.4f}")
```

```
print(f"beta = {beta:.4f}")
print(f"R² = {r_squared:.4f}")
```

SCALING MODEL

```
=====
alpha = 16.0418
beta = 0.9208
R² = 0.7628
```

```
In [47]: # =====
# BLOCK 6E – CREATE LOCKED TEST SET
# =====

from sklearn.model_selection import train_test_split

features_v01 = [
    "powerCapacityMW",
    "totalSquareFeet"
]

experiment_data = cost_data[
    cost_data[features_v01].notna().all(axis=1)
    & (cost_data["powerCapacityMW"] > 0)
    & (cost_data["totalSquareFeet"] > 0)
    & (cost_data["investmentPerMW"] > 0)
].copy()

development_data, test_data = train_test_split(
    experiment_data,
    test_size=0.20,
    random_state=42
)

print("EXPERIMENTAL SPLIT")
print("=" * 50)

print(
    f"Total:      {len(experiment_data):,}"
)

print(
    f"Development: {len(development_data):,}"
)

print(
    f"Locked test: {len(test_data):,}"
)
```

EXPERIMENTAL SPLIT

```
=====
Total:      503
Development: 402
Locked test: 101
```

```
In [48]: # =====
# BLOCK 6G – CROSS-VALIDATED k SELECTION
# =====

from sklearn.model_selection import KFold

def evaluate_k_cv(
    data,
    k_values,
    n_splits=5,
```

```

random_state=42
):

features = [
    "powerCapacityMW",
    "totalSquareFeet"
]

kfold = KFold(
    n_splits=n_splits,
    shuffle=True,
    random_state=random_state
)

all_results = []

for k in k_values:

    fold_maes = []

    for train_idx, val_idx in kfold.split(data):

        train = data.iloc[
            train_idx
        ].copy()

        validation = data.iloc[
            val_idx
        ].copy()

        # -----
        # TRAINING FEATURES
        # -----

        X_train = train[
            features
        ]

        y_train = train[
            "investmentPerMW"
        ].to_numpy()

        # -----
        # VALIDATION FEATURES
        # -----

        X_val = validation[
            features
        ]

        y_val = validation[
            "investmentPerMW"
        ].to_numpy()

        # -----
        # FIT SCALER ONLY ON TRAINING DATA
        # -----

        scaler = StandardScaler()

        X_train_scaled = scaler.fit_transform(
            X_train
        )

```

```

X_val_scaled = scaler.transform(
    X_val
)

# -----
# FIT NEIGHBOUR MODEL ON TRAINING ONLY
# -----

knn = NearestNeighbors(
    n_neighbors=k,
    metric="euclidean"
)

knn.fit(
    X_train_scaled
)

_, neighbour_indices = (
    knn.kneighbors(
        X_val_scaled
    )
)

# -----
# PREDICT VALIDATION PROJECTS
# -----

predictions = []

for neighbours in neighbour_indices:

    neighbour_costs = y_train[
        neighbours
    ]

    predictions.append(
        np.median(
            neighbour_costs
        )
    )

predictions = np.array(
    predictions
)

# -----
# VALIDATION LOSS
# -----

fold_mae = mean_absolute_error(
    y_val,
    predictions
)

fold_maes.append(
    fold_mae
)

# -----
# STORE k PERFORMANCE
# -----

all_results.append({
    "k": k,

```

```

        "mean_cv_mae": np.mean(
            fold_maes
        ),
        "std_cv_mae": np.std(
            fold_maes
        )
    })

    return pd.DataFrame(
        all_results
    ).sort_values(
        "mean_cv_mae"
    )

```

```

In [49]: # =====
# BLOCK 6H – k TOURNAMENT
# =====

k_values = [
    3,
    5,
    10,
    15,
    20,
    30,
    50,
    75,
    100
]

k_results = evaluate_k_cv(
    development_data,
    k_values
)

display(k_results)

```

	k	mean_cv_mae	std_cv_mae
8	100	5.879043e+06	1.508250e+06
7	75	5.893647e+06	1.523022e+06
6	50	5.907266e+06	1.540936e+06
5	30	5.929706e+06	1.537006e+06
3	15	5.946536e+06	1.563211e+06
4	20	5.975747e+06	1.551079e+06
2	10	6.148467e+06	1.512130e+06
1	5	6.389182e+06	1.764021e+06
0	3	6.793727e+06	1.722174e+06

```

In [50]: # =====
# BLOCK 6I – REFERENCE-CLASS SIZE VS ERROR
# =====

plot_data = k_results.sort_values(
    "k"
)

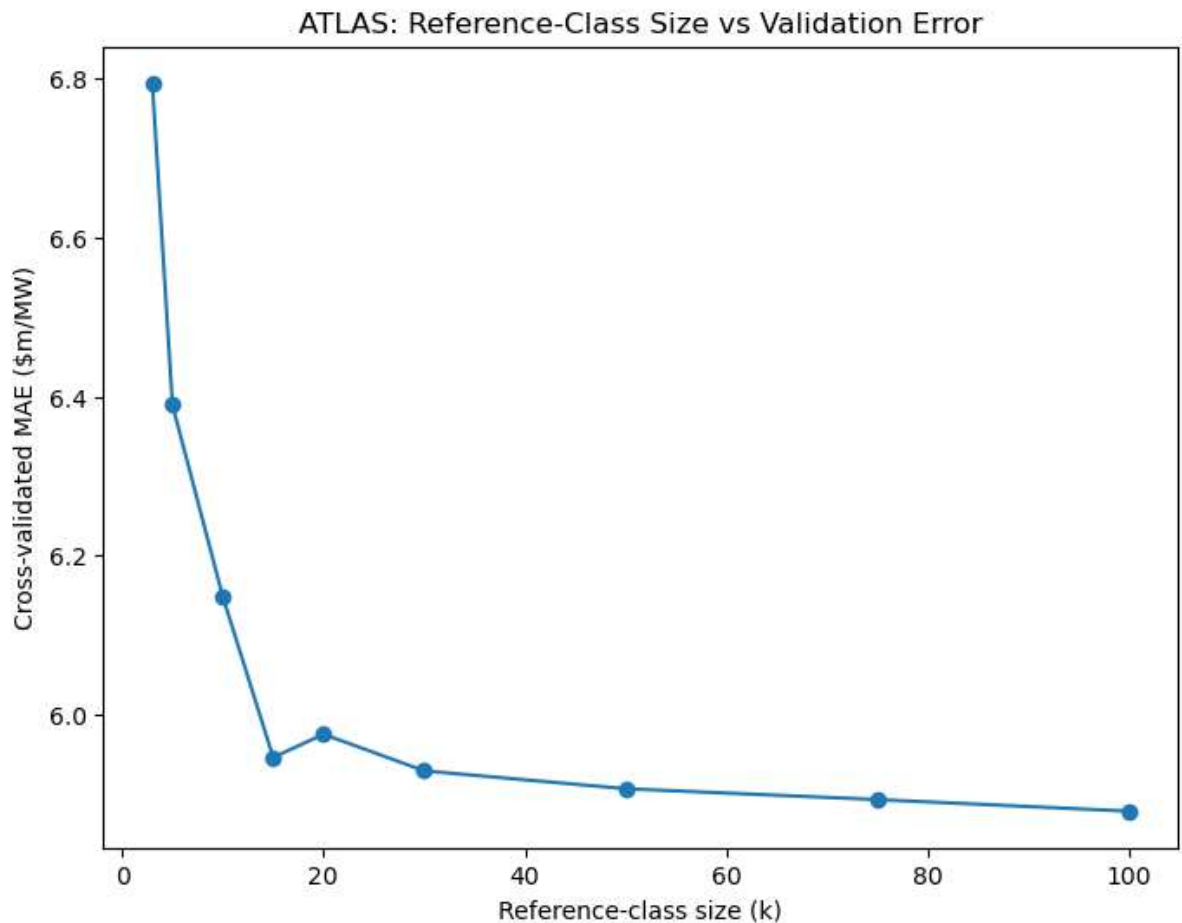
```

```
plt.figure(figsize=(8, 6))

plt.plot(
    plot_data["k"],
    plot_data["mean_cv_mae"] / 1_000_000,
    marker="o"
)

plt.xlabel("Reference-class size (k)")
plt.ylabel("Cross-validated MAE ($m/MW)")
plt.title("ATLAS: Reference-Class Size vs Validation Error")

plt.show()
```



```
In [51]: # =====
# BLOCK 6J – CROSS-VALIDATED GLOBAL BASELINE
# =====

kfold = KFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

global_fold_maes = []

for train_idx, val_idx in kfold.split(
    development_data
):

    train = development_data.iloc[
        train_idx
    ]
```

```

validation = development_data.iloc[
    val_idx
]

global_prediction = train[
    "investmentPerMW"
].median()

predictions = np.full(
    len(validation),
    global_prediction
)

fold_mae = mean_absolute_error(
    validation["investmentPerMW"],
    predictions
)

global_fold_maes.append(
    fold_mae
)

global_cv_mae = np.mean(
    global_fold_maes
)

print(
    f"Global CV MAE: "
    f"${global_cv_mae/1e6:,.2f}m/MW"
)

print(
    f"Best RCF CV MAE: "
    f"${k_results.iloc[0]['mean_cv_mae']/1e6:,.2f}m/MW"
)

```

Global CV MAE: \$5.86m/MW
 Best RCF CV MAE: \$5.88m/MW

```

In [53]: # =====
# BLOCK 6K - FREEZE DEVELOPMENT RESULT
# =====

# Identify the k with the lowest cross-validated MAE
best_row = k_results.loc[
    k_results["mean_cv_mae"].idxmin()
]

best_k = int(
    best_row["k"]
)

best_rcf_cv_mae = float(
    best_row["mean_cv_mae"]
)

# Calculate improvement relative to global baseline
cv_improvement = (
    (global_cv_mae - best_rcf_cv_mae)
    / global_cv_mae
) * 100

# Freeze ATLAS v0.2 development specification

```

```

atlas_v02_spec = {

    "target": "investmentPerMW",

    "features": [
        "powerCapacityMW",
        "totalSquareFeet"
    ],

    "scaling": "StandardScaler",

    "distance": "Euclidean",

    "estimator": "median",

    "selected_k": best_k,

    "selection_method":
        "5-fold development-set cross-validation",

    "primary_metric": "MAE"
}

# -----
# REPORT FROZEN SPECIFICATION
# -----

print("=" * 60)
print("ATLAS RCF v0.2 - DEVELOPMENT SPECIFICATION")
print("=" * 60)

print(f"\nSelected k:          {best_k}")

print(
    f"Best RCF CV MAE:          "
    f"${best_rcf_cv_mae/1e6:,.2f}m/MW"
)

print(
    f"Global CV MAE:            "
    f"${global_cv_mae/1e6:,.2f}m/MW"
)

print(
    f"MAE change vs global:    "
    f"${cv_improvement:+.1f}%"
)

print("\nFROZEN SPECIFICATION")
print("-" * 60)

for key, value in atlas_v02_spec.items():
    print(f"{key}: {value}")

```

```
=====
ATLAS RCF v0.2 – DEVELOPMENT SPECIFICATION
=====
```

```
Selected k:          100
Best RCF CV MAE:     $5.88m/MW
Global CV MAE:       $5.86m/MW
MAE change vs global: -0.3%
```

```
FROZEN SPECIFICATION
-----
```

```
target: investmentPerMW
features: ['powerCapacityMW', 'totalSquareFeet']
scaling: StandardScaler
distance: Euclidean
estimator: median
selected_k: 100
selection_method: 5-fold development-set cross-validation
primary_metric: MAE
```

```
In [54]: k_results["mean_cv_mae"].idxmin()
```

```
Out[54]: 8
```

```
In [55]: # =====
# BLOCK 7B – FINAL LOCKED TEST
# =====

def final_locked_test(
    development_data,
    test_data,
    k
):

    features = [
        "powerCapacityMW",
        "totalSquareFeet"
    ]

    # -----
    # TRAIN / TEST DATA
    # -----

    X_train = development_data[features]
    y_train = development_data[
        "investmentPerMW"
    ].to_numpy()

    X_test = test_data[features]
    y_test = test_data[
        "investmentPerMW"
    ].to_numpy()

    # -----
    # PREPROCESSING
    # Fit ONLY on development data
    # -----

    scaler = StandardScaler()

    X_train_scaled = scaler.fit_transform(
        X_train
    )
```

```
X_test_scaled = scaler.transform(
    X_test
)

# -----
# FROZEN kNN REFERENCE CLASS
# -----

knn = NearestNeighbors(
    n_neighbors=k,
    metric="euclidean"
)

knn.fit(
    X_train_scaled
)

distances, indices = knn.kneighbors(
    X_test_scaled
)

rcf_predictions = []

for neighbours in indices:
    neighbour_costs = y_train[
        neighbours
    ]

    rcf_predictions.append(
        np.median(neighbour_costs)
    )

rcf_predictions = np.array(
    rcf_predictions
)

# -----
# GLOBAL OUTSIDE-VIEW BASELINE
# -----

global_median = np.median(
    y_train
)

global_predictions = np.full(
    len(y_test),
    global_median
)

# -----
# PERFORMANCE
# -----

global_mae = mean_absolute_error(
    y_test,
    global_predictions
)

rcf_mae = mean_absolute_error(
    y_test,
    rcf_predictions
)
```

```
global_rmse = np.sqrt(
    mean_squared_error(
        y_test,
        global_predictions
    )
)

rcf_rmse = np.sqrt(
    mean_squared_error(
        y_test,
        rcf_predictions
    )
)

mae_change = (
    (global_mae - rcf_mae)
    / global_mae
) * 100

# -----
# PROJECT-LEVEL RESULTS
# -----

results = test_data[
    [
        "projectName",
        "powerCapacityMW",
        "totalSquareFeet",
        "investmentPerMW"
    ]
].copy()

results["global_prediction"] = (
    global_predictions
)

results["rcf_prediction"] = (
    rcf_predictions
)

results["global_absolute_error"] = np.abs(
    y_test - global_predictions
)

results["rcf_absolute_error"] = np.abs(
    y_test - rcf_predictions
)

return {

    "n_train": len(development_data),
    "n_test": len(test_data),

    "k": k,

    "global_mae": global_mae,
    "rcf_mae": rcf_mae,

    "global_rmse": global_rmse,
    "rcf_rmse": rcf_rmse,

    "mae_change": mae_change,

    "results": results
}
```

```

    }

    final_test = final_locked_test(
        development_data,
        test_data,
        atlas_v02_spec["selected_k"]
    )

```

```

In [56]: # =====
# BLOCK 7C - FINAL RESULT
# =====

print("=" * 60)
print("ATLAS RCF - FINAL LOCKED TEST")
print("=" * 60)

print(
    f"Development projects:  "
    f"{final_test['n_train']}"
)

print(
    f"Unseen test projects:  "
    f"{final_test['n_test']}"
)

print(
    f"Frozen k:              "
    f"{final_test['k']}"
)

print("\nGLOBAL OUTSIDE VIEW")
print("-" * 60)

print(
    f"MAE:                    "
    f"${final_test['global_mae']/1e6:,.2f}m/MW"
)

print(
    f"RMSE:                   "
    f"${final_test['global_rmse']/1e6:,.2f}m/MW"
)

print("\nML-SELECTED REFERENCE CLASS")
print("-" * 60)

print(
    f"MAE:                    "
    f"${final_test['rcf_mae']/1e6:,.2f}m/MW"
)

print(
    f"RMSE:                   "
    f"${final_test['rcf_rmse']/1e6:,.2f}m/MW"
)

print("\nPRIMARY RESULT")
print("-" * 60)

print(
    f"MAE change vs global:  "

```

```
f"{final_test['mae_change']:+.2f}%"
)
```

=====

ATLAS RCF – FINAL LOCKED TEST

=====

Development projects: 402
 Unseen test projects: 101
 Frozen k: 100

GLOBAL OUTSIDE VIEW

MAE: \$6.59m/MW
 RMSE: \$17.60m/MW

ML-SELECTED REFERENCE CLASS

MAE: \$6.62m/MW
 RMSE: \$17.50m/MW

PRIMARY RESULT

MAE change vs global: -0.56%

```
In [58]: # =====
# BLOCK 7D – FINAL PUBLICATION FIGURE WITH DATA LABELS
# =====

model_names = [
    "Global outside view",
    "ML-selected reference class"
]

mae_values = [
    final_test["global_mae"] / 1e6,
    final_test["rcf_mae"] / 1e6
]

mae_change = final_test["mae_change"]

# -----
# CREATE FIGURE
# -----

fig, ax = plt.subplots(figsize=(8, 6))

bars = ax.bar(
    model_names,
    mae_values
)

# -----
# ADD EXACT DATA LABELS
# -----

for bar, value in zip(bars, mae_values):
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height(),
        f"${value:.2f}m/MW",
        ha="center",
        va="bottom",
        fontsize=12,
```

```
        fontweight="bold"
    )

# -----
# TITLES AND LABELS
# -----

ax.set_ylabel(
    "Mean Absolute Error ($m/MW)"
)

ax.set_title(
    "ATLAS RCF: Locked Out-of-Sample Performance",
    fontsize=14,
    fontweight="bold"
)

fig.text(
    0.5,
    0.91,
    f"RCF MAE change relative to global baseline: {mae_change:+.2f}%",
    ha="center",
    fontsize=10
)

# Give Labels some space above bars
ax.set_ylim(
    0,
    max(mae_values) * 1.15
)

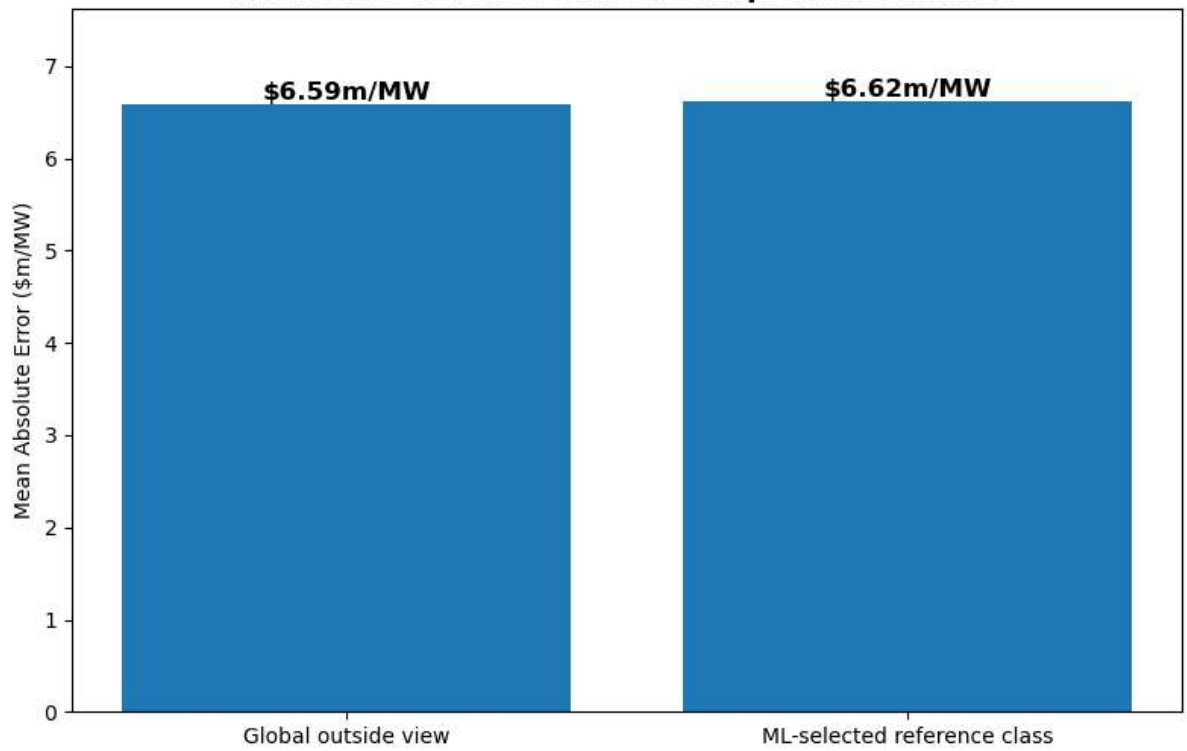
# -----
# PUBLICATION OUTPUT
# -----

plt.tight_layout(
    rect=[0, 0, 1, 0.90]
)

plt.savefig(
    "atlas_final_test.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```

RCF MAE change relative to global baseline: -0.56%

ATLAS RCF: Locked Out-of-Sample Performance

In []: